What is Cross-Validation?

Testing your machine learning models with cross-validation

Mohammed Alhamid Follow 8 min read · Dec 25, 2020

184

Photo by Joshua Sortino on Unsplash

Cross-Validation (CV) is one of the key topics around testing your learning models. Although the subject is widely known, I still find some misconceptions cover some of its aspects. When we train a model, we split the dataset into two main sets: training and testing. The training set represents all the examples that a model is learning from, while the testing set simulates the testing examples as in Figure 1.

Figure 1: Typical dataset split into training and testing sets.

Why cross-validation?

> *CV provides the ability to estimate model performance on unseen data not used while training.*

Data scientists rely on several reasons for using cross-validation during their building process of Machine Learning (ML) models. For instance, tuning the model hyperparameters, testing different properties of the overall datasets, and iterate the training process. Also, in cases where your training dataset is small, and the ability to split them into training, validation, and testing will significantly affect training accuracy. The following main points can

summarize the reason we use a CV, but they overlap. Hence, the list is presented here in a simplified way:

(1) Testing on unseen data

One of the critical pillars of validating a learning model before putting them in production is making accurate predictions on unseen data. The unseen data is all types of data that a model has never learned before. Ideally, the testing data is supposed to flow directly to the model in many testing iterations. However, in reality, access to such data is limited or not yet available in a new environment.

The typical 80–20 rule of splitting data into training and testing can still be vulnerable to accidentally ending up in a perfect split that boosts the model accuracy while limiting it from performing the same in a real environment. Sometimes, the accuracy calculated this way is mostly a matter of luck! The 80–20 is not an actual rule per se, and you will find alternative ratios that range between 25~30% for testing and 70~75% for training.

(2) Tuning model hyperparameter

Finding the best combination of model parameters is a common step to tune an algorithm toward learning the dataset's hidden patterns. But, doing this step on a simple training-testing split is typically not recommended. The model performance is usually very sensitive to such parameters, and adjusting those based on a predefined dataset split should be avoided. It can cause the model to overfit and reduce its ability to generalize.

(3) The third split is not achievable

For parameter tuning and avoid model overfitting, you might find some recommendations around splitting the dataset into three partitions: training, testing, and validation. For instance, 70% of the data is used for training, 20% for validation, and the remaining 10% is used for testing. In cases where the actual dataset is small, we might need to invest in using the maximum amount of data to train the model. In other instances, splitting into such three partitions could create bias in the training process where some

significant examples are kept in training or validation splits. Hence, CV can become handy to resolve this issue.

(4) Avoid instability of sampling

Sometimes the splits of training-testing data can be very tricky. The properties of the testing data are not similar to the properties of the training. Although randomness ensures that each sample can have the same chance to be selected in the testing set, the process of a single split can still bring instability when the experiment is repeated with a new division.

How does it work?

Cross-Validation has two main steps: splitting the data into subsets (called folds) and rotating the training and validation among them. The splitting technique commonly has the following properties:

- Each **fold** has approximately the same size.
- Data can be **randomly** selected in each fold or **stratified**.
- All folds are used to train the model except one, which is used for validation. That validation fold should be rotated until all folds have become a validation fold once and only once.
- Each example is recommended to be contained in one and only one fold.

K-fold and **CV** are two terms that are used interchangeably. K-fold is just describing how many folds you want to split your dataset into. Many libraries use k=10 as a default value representing 90% going to training and 10% going to the validation set. The next figure describes the process of iterating over the picked ten folds of the dataset.

	Fold-1	Fold-2	Fold-3	Fold-4	Fold-5	Fold-6	Fold-7	Fold-8	Fold-9	Fold-10
Step-1	Train	Train	Train	Train	Train	Train	Train	Train	Train	Test
Step-2	Train	Train	Train	Train	Train	Train	Train	Train	Test	Train
Step-3	Train	Train	Train	Train	Train	Train	Train	Test	Train	Train
Step-4	Train	Train	Train	Train	Train	Train	Test	Train	Train	Train
Step-5	Train	Train	Train	Train	Train	Test	Train	Train	Train	Train
Step-6	Train	Train	Train	Train	Test	Train	Train	Train	Train	Train
Step-7	Train	Train	Train	Test	Train	Train	Train	Train	Train	Train
Step-8	Train	Train	Test	Train	Train	Train	Train	Train	Train	Train
Step-9	Train	Test	Train	Train	Train	Train	Train	Train	Train	Train
Step-10	Test	Train	Train	Train	Train	Train	Train	Train	Train	Train

Figure 2: A 10-fold representation of how each fold is used in the cross-validation process.

Figure 2 shows how one fold in each step iteratively is held out for testing while the remaining folds are used to build the model. Each step calculates the prediction error. Upon the completion of the final step, a list of computed error are produced of which we can take the mean.

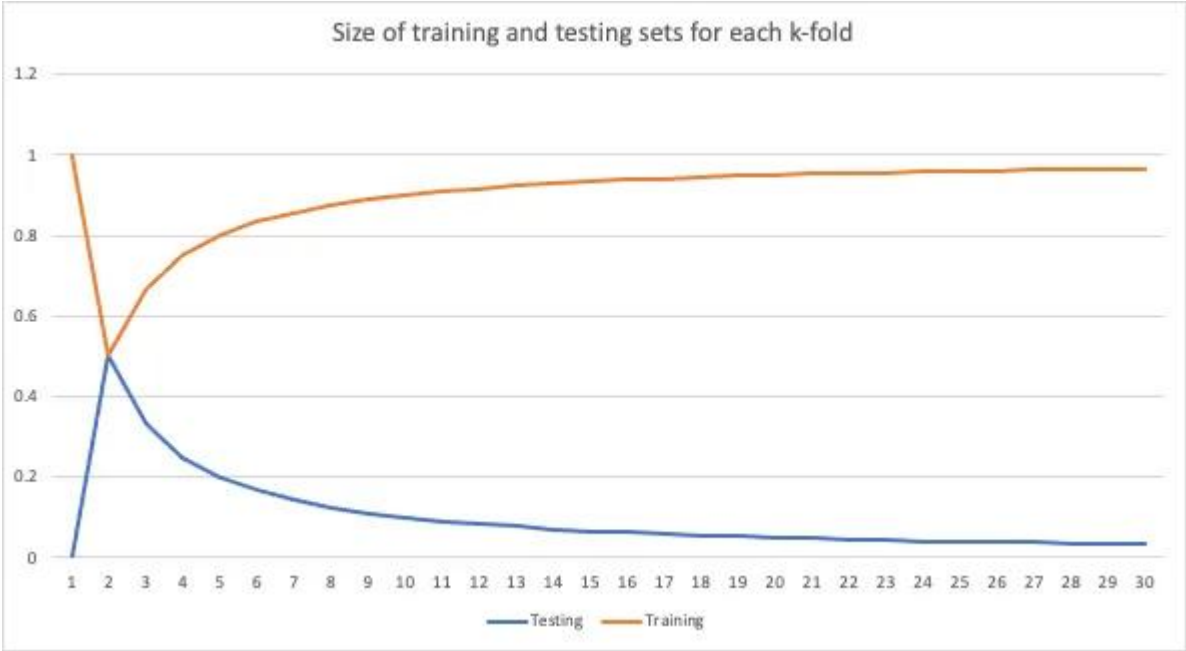


Figure 3: A graph representation showing training vs. testing size in each value picked for k.

Figure 3 shows the change in the training and validation sets' size when using different values for k. The training set size increases whenever we increase the number of folds while the validation set decreases. Typically, the smaller the validation set, the likelihood that the randomness rises with

a high noise variation. Increasing the training set size will increase such randomness and bring more reliable performance metrics.

Analyzing the results

One of the advantages of CV is observing the model predictions against all the instances in the dataset. It ensures that the model has been tested on the full data without testing them simultaneously. Variations are expected in each step of the validation; therefore, computing the mean and standard deviation can reduce the information into a few comparing values.

Other techniques for cross-validation

There are other techniques on how to implement cross-validation. Let's jump into some of those:

This approach is quite expensive and requires each holdout example to be tested using a model. It also might increase the overall error rate and becomes computationally costly if the dataset size is large. Usually, it is recommended when the dataset size is small. Figure 4 demonstrates the overall process of a simple dataset containing ten examples.

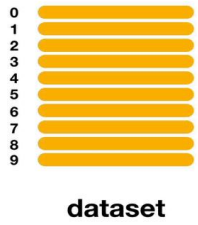


Figure 4: A demonstration of how the dataset is split into partitions using LOOCV.

(2) Leave-pair-out cross-validation (LPOCV)

It is another exhaustive technique for performing CV. We can define in each iteration how many examples shall be used for testing the model and how many shall be left for training. The process is repeated for all possible combinations of pairs in the dataset. Note that LOOCV is a LOPCV technique where p =1.

Example

Let's refresh our minds on how to split the data using the Sklearn library. The following code divides the dataset into two splits: training and testing. We defined here that 1/3 of the dataset should be used for testing.

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.33, random_state=1)
```

We will then build a function to show how k-fold works compared to the single split in the previous code.

```
def k_Fold_Split(data, n_splits, shuffle, random_state,
verbose=False):
    # Creating k-fold splitting
    kfold = KFold(n_splits, shuffle, random_state)
    sizes = [0 , 0]
    for train, test in kfold.split(data):
        if verbose:
            print('train: indexes %s, val: indexes %s, size
(training) %d, (val) %d' % (train, test, train.shape[0],
test.shape[0]))
        sizes[0] += train.shape[0]
        sizes[1] += test.shape[0]

    return int(sizes[0]/n_splits), int(sizes[1]/n_splits)
```

The `k_fold_split` function shows the indexes and the size of each partition. It enables you to track how much data is dedicated for each fold and the chosen indexes.

LOOCV

Using LOOCV as a splitting strategy is pretty straight forward. We will use again Sklearn library to perform the cross-validation.

```
from sklearn.model_selection import LeaveOneOut
cv_strategy = LeaveOneOut()
# cross_val_score will evaluate the model
scores = cross_val_score(estimator, X, y, scoring='accuracy',
cv=cv_strategy, n_jobs=-1)
```

Careful Considerations

Time-series dataset

Cross-validation is a great way to ensure the training dataset does not have an implicit type of ordering. However, some cases require the order to be preserved, such as time-series use cases. We can still use cross-validation for time-series datasets using some other technique such as time-based folds.

Unbalanced dataset

Dealing with cross-validation in an unbalanced dataset can be tricky as well. If oversampling is used, the leaking of the oversampled examples can mislead the CV results. One way to consider is the use of stratified sampling instead of splitting randomly. Here is a fantastic blog article discussing how to handle this situation.

Nested cross-validation

CV measures the generalization of the model, but it cannot avoid bias altogether. Nested cross-validation focuses on ensuring the model's hyperparameters are not overfitting the dataset. The nested keyword comes to hint at the use of double cross-validation on each fold. The hyperparameter tuning validation is achieved using another k-fold splits on the folds used to train the model.

Overfitting

As mentioned earlier, CV is used to measure if a learning model can **generalize** on unseen data. We rely on using a validation dataset for early stopping and parameter tuning different from the testing set during the building of models. There are a couple of research papers recently suggesting the use of CV to measure overfitting as well. In this case, the direct application would be the use of CV as a validation set for a learning model.

Summary

Cross-validation is a procedure to evaluate the performance of learning models. Datasets are typically split in a random or stratified strategy. The splitting technique can be varied and chosen based on the data's size and the ultimate objective. Also, the computational cost plays a role in implementing the CV technique. Regression and classification problems can use CV, but careful consideration must be paid for some types of datasets such as time-series. A complete example of the code can be found on my Github.

Thank you!

Machine Learning | K Fold Cross Validation | Crossvalidation

Published in TDS Archive

822K followers · Last published Feb 4, 2025

Follow

An archive of data science, data analytics, data engineering, machine learning, and artificial intelligence writing from the former Towards Data Science Medium publication.

Written by Mohammed Alhamid

280 followers · 10 following

Follow

PhD, working on Machine Learning and AI

No responses yet

Write a response

What are your thoughts?

More from Mohammed Alhamid and TDS Archive

In TDS Archive by Mohammed Alhamid

Python Numba or NumPy: understand the differences

Short description supported by examples.

Feb 28, 2020 107

In TDS Archive by Luís Roque

Agentic AI: Building Autonomous Systems from Scratch

A Step-by-Step Guide to Creating Multi-Agent Frameworks in the Age of Generative AI

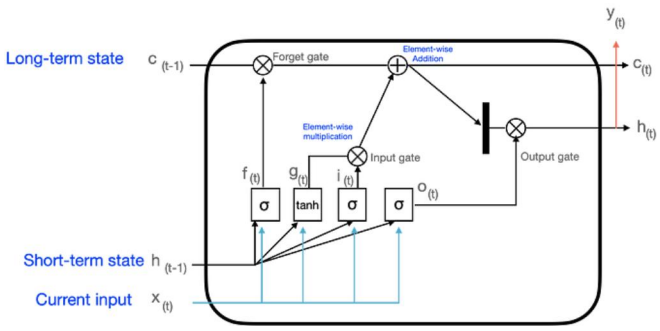
Dec 13, 2024 1.1K 24

In TDS Archive by Thuwarakesh Murallie

How to Build a Knowledge Graph in Minutes (And Make It Enterprise-...

I tried and failed creating one—but it was when LLMs were not a thing!

Jan 13 1.1K 8



In TDS Archive by Mohammed Alhamid

LSTM and Bidirectional LSTM for Regression

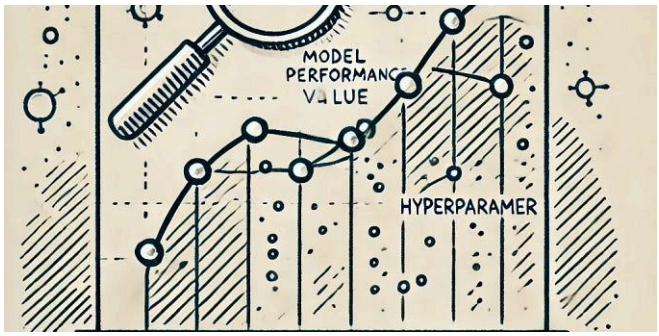
Learn how to use Long Short-Term Memory Networks for regression problems

Jun 27, 2021 85 1

See all from Mohammed Alhamid

See all from TDS Archive

Recommended from Medium

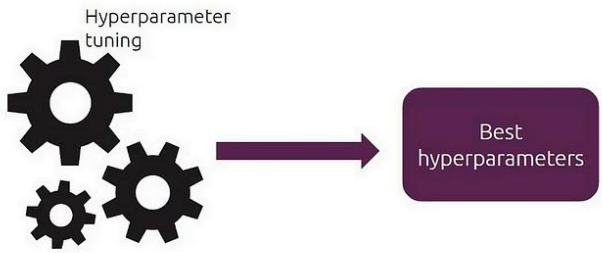


In The Machine ... by Ranjeet Tiwari | Senior Arc...

Mastering Hyperparameter Tuning: Unlocking Machine Learning...

Imagine you're baking a cake 🍰. You have the perfect recipe, but what if the oven...

Jan 5 100



Abhay singh

Hyperparameter Tuning: Beyond the Basics

Hyperparameters are external configuration variables that data scientists use to manage...

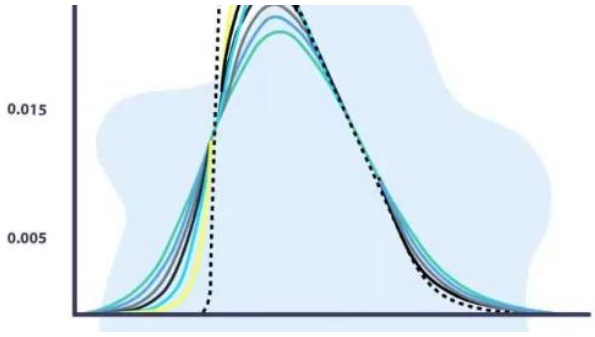
Dec 10, 2024 19

Damini Vadrevu

NumPy Arrays in Machine Learning

There's no modeling without them.

May 16 1

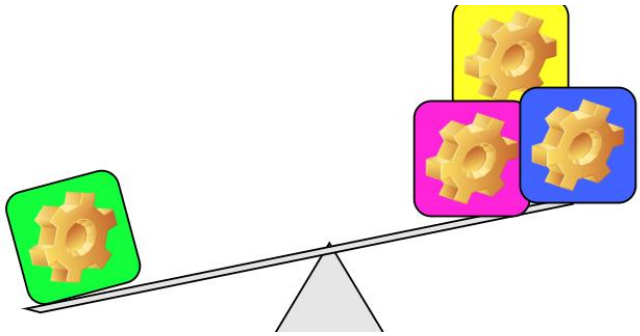


Shridhar Pawar

Understanding JS Divergence for Feature Selection: A Hands-On...

Feature selection is a critical step in building robust machine learning models. One...

Feb 28 50

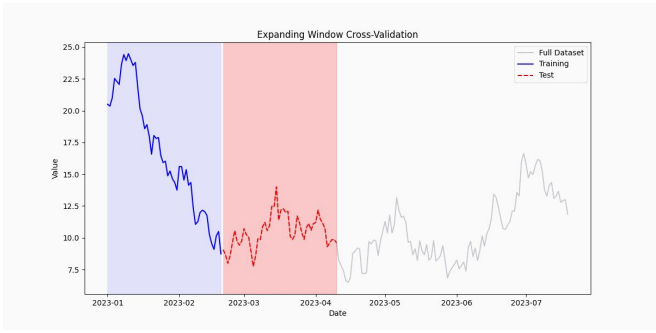


Nermin Babalık

Handling Imbalanced Dataset in Deep Learning

Let’s consider undersampling, oversampling, SMOTE and ensemble methods for...

Jan 3 403 2



Kyle Jones

Cross-Validation for Time Series Data

Time series cross-validation differs fundamentally from standard cross-validatio...

Jan 18 96 2

See more recommendations